

I/O Yonetimi

Input/Output Management

Kapsamli Ders Notu – Sınay Odakli

Ders 8

Ele Alınan Konular

- I/O Modelleri (Unix, Windows, Ag Aygıtları) • Aygt Surucusu Mimarisi
- Programmed I/O, Interrupt-Driven I/O, DMA • Bellek-Eslemeli I/O
- Kesme Yonetimi (Precise/Imprecise) • I/O Yazılım Katmanları
- Arabellekleme (Buffering) • Disk Zamanlama (FCFS, SSF, Elevator)
- RAID, CD/DVD, Disk Bicimlendirme, Hatalar • Sabit Depolama
- Saat, Klavye, X Window, Güç Yonetimi • Daemon ve Sistem Baslatma

İçindekiler

1	I/O ve İşletim Sistemi	4
1.1	Temel Kavramlar	4
1.2	I/O Modelleri	4
1.3	Unix Aygıt Dosyaları: /dev, Major/Minor Numaralar	4
1.4	Aygıtla Özgü Semantikler ve ioctl	5
2	Aygıt Surucusu Mimarisi	5
2.1	Surucunun Standart Giriş Noktaları	5
2.2	Block Aygıtlar ve Buffer Cache	5
2.3	Uniform Arayüz: Standart Surucusuz vs Standartlı	6
3	Grafik Aygıtları	6
3.1	Basit Bitmap Modeli ve Donanım Yardımı	6
3.2	Pencereler ve Programlama Arayüzü	6
4	Ağ Aygıtları (Network Devices)	6
4.1	Temel Farklar	6
4.2	Ağ Çıkışı	7
4.3	Ağ Girişi	7
4.4	Ağ Yığını (Network Stack)	7
5	Gerçek I/O Sorunları ve Surucu Örnekleri	7
5.1	Fiziksel Dünya Tuhaflıkları	7
5.2	Linux Yazıcı Surucusu Yorumu (drivers/char/lp.c)	8
5.3	NetBSD Yazıcı Surucusu Test Yorumu	8
5.4	Hata Kurtarma Kodunu Test Etmek (Slayt 67)	8
6	I/O Gerçekleştirme Yöntemleri	8
6.1	Programmed I/O (Dönüşel Sorgulama – Polling)	8
6.2	Interrupt-Driven I/O (Kesme Tabanlı)	9
6.3	DMA (Direct Memory Access – Doğrudan Bellek Erişimi)	9
7	Bellek-Eslemeli I/O (Memory-Mapped I/O) ve IOMMU	10
7.1	MMIO Mimarileri	10
7.2	IOMMU	10
8	Kesme Yönetimi (Interrupt Management)	10
8.1	Kesme Nasıl Olur?	10
8.2	Precise vs Imprecise Interrupts	11
8.3	Kesme Yuku ve Coalescing	11
9	I/O Yazılım Katmanları	11
9.1	Kesme İşleyici – 10 Adım	12
9.2	Aygıt-Bağımsız I/O Yazılımı	12
9.3	Arabellekleme (Buffering)	12
10	Bellek Erişim Sorunları ve DMA	13
10.1	Sanal Bellek Sorunu	13
10.2	Scatter/Gather I/O	13

10.3 Direct I/O ve Sayfa Kilitleme	13
10.4 Bounce Buffer	13
10.5 Disk Blok Adres Uzayı – Tarihsel Sinirlar	14
11 Disk Zamanlama Algoritmaları	14
11.1 Disk Erisim Suresi Bilesenleri	14
11.2 Catısan Gereksinimler	14
11.3 FCFS (First Come, First Served)	14
11.4 SSF (Shortest Seek First)	14
11.5 Elevator (SCAN) Algoritması	14
11.6 Donme Gecikmesi ve Silindir Eskiltmesi (Cylinder Skew)	15
12 Manyetik Disk Parametreleri ve Bicimlendirme	15
12.1 Disk Parametreleri	15
12.2 Sektor Yapısı	16
13 Hata Isleme ve Sabit Depolama	16
13.1 Hata Tipleri	16
13.2 Sabit Depolama (Stable Storage)	16
14 RAID	17
15 Optik Depolama: CD-ROM, CD-R, DVD	17
16 Saat Donanimi ve Yazilimi	17
16.1 Saat Donanimini	17
16.2 Saat Surucusunun Sorumluluklar	18
16.3 Gunun Saatini Tutma	18
16.4 Tek Saat ile Birden Fazla Timer	18
16.5 Soft Timer	18
17 Klavye ve Terminal Yazilimi	18
18 X Window Sistemi ve GUI	19
18.1 Mimari	19
18.2 GUI: X Window Uygulama Yapısı	19
18.3 GUI: Windows Ana Program Yapısı	19
18.4 Bitmap Islemleri	20
18.5 Fontlar	20
18.6 Thin Client (Ince Istemci)	20
19 Guc Yonetimi (Power Management)	20
19.1 Donanim Guc Tuketimi	20
19.2 Ekran Bolge (Zone) Yonetimi	21
19.3 CPU Guc Olckeleme – DVFS	21
20 Kernel vs. Kullanici Uzayi Karari	21
20.1 Kernel'e Koyma Kriterleri	21
20.2 Kullanici Uzayina Koyma Kriterleri	21
21 Daemon'lar ve Guvenlik	22

21.1 Daemon Nedir?	22
21.2 Yazici Daemon Ornegi	22
21.3 Yazici Daemon'da Guvenlik	22
21.4 Daemon Izinleri: En Az Ayricalik Prensibi	23
22 NFS ve Dosya Kilitleme	23
23 Sistem Baslatma: init, Calistirma Duzeyleri	24
23.1 /sbin/init – PID 1	24
23.2 Sistem Ayarlama (sysinit)	24
23.3 Calistirma Duzeyleri (Run Levels)	24
23.4 /etc/rcN.d Betikleri	24
23.5 Diger Hayati Komutlar	24
24 Uygulama Problemleri – Sinav Hazirlik	25
24.1 Problem 1: DMA Transfer Suresi Hesabi	25
24.2 Problem 2: Disk Erisim Suresi	25
24.3 Problem 3: Disk Zamanlama Karsistirma	25
24.4 Problem 4: Precise Interrupt Analizi	26
24.5 Problem 5: IOMMU vs Bounce Buffer	26
24.6 Problem 6: Yazici Daemon Guvenlik Analizi	26
25 Karsilastirma Tablolari	26
25.1 I/O Yazilim Katmanlari Ozeti	26
25.2 RAID Secimleri	26
25.3 I/O Yontemleri Ozeti	27
26 Kavram Sozlugu	27
27 Sinav Calistirma – Son Revizyon Listesi	28

1 I/O ve Isletim Sistemi

1.1 Temel Kavramlar

Bir bilgisayar sisteminde tumt I/O islemleri isletim sistemi uzerinden gecmek zorundadir. Isletim sistemi olmadan her uygulamanin disk, printer, ag karti gibi donanimi dogrudan kontrol etmesi gerekir; bu durum cakismalara, guvenlik aciklarina ve kargasaya yol acar. OS bu kargasayi onlemek icin aygitlari uzerinde bir soyutlama katmani olusturur.

[*] Onemli

Bir sistemde klavyeden disk surucusune, ag kartindan ekran kartina kadar onlarca farkli turde aygit bulunabilir. Bu aygitlari her biri farkli hizlarda, farkli protokollerle ve farkli hata modlariyla calisir. OS tasarimcisinin gorevleri: (1) aygitlarla konusan yazilimi (device driver) tanimlamak, (2) OS-driver arayuzunu belirlemek, (3) uygulama-OS arayuzunu belirlemek ve (4) donanim kisitlamalarini cozmek.

1.2 I/O Modelleri

Isletim sistemleri aygitlari farkli bicimde modeller. Ana yaklasimlar:

Model	Tanim	Ornekler
Unix: Block	Sabit boyutlu bloklara gore okuma/yazma	Disk, SSD, USB depolama
Unix: Character	Yapisiz bayt akisi	Terminal, fare, ses karti
Windows: Bitmap	Ekran/yazici icin piksel dizisi	Ekran, yazici
Network	Paket tabanli, ne block ne character	Ethernet, Wi-Fi

Unix Modeli: Unix iki kategoriye ayirir: *block devices* ve *character devices*. Dosya sistemleri block devices uzerinde yasarken, terminal, fare gibi aygitlar character devices olarak islenir. Bu yaklasim uygulamalari buyuk cogunlugu icin *device independence* (aygit bagimsizligi) saglar.

Windows Modeli: Bitmap yönelimlidir. Ekran ve yazici piksel dizisi (bitmap) seklinde modellenirken diskler yine block modelini kullanir.

Ag Aygitlari: Ne block ne de character ozelligindedir. Peket tabanlidir; OS ile cok farkli bicimde etkilesir.

1.3 Unix Aygit Dosyalari: /dev, Major/Minor Numaralar

Unix'te tumt aygitlari dosya isimleri vardir; bunlar /dev dizininde bulunur.

Listing 1: Aygit dosyasi listeleme ornegi

```
1 $ ls --time-style "+" -Ggl /dev/sda1 /dev/pts/6
2 crw--w---- 1 136, 6 /dev/pts/6
3 brw-rw---- 1 8, 1 /dev/sda1
```

- c – character device; b – block device
- **Major device number:** Hangi surucunun kullanilacagini gosterir (ornegin 8 = SCSI disk). Block ve character aygitlar icin ayri ad uzaylaridir.
- **Minor device number:** Hangi birim oldugunu gosterir (ornegin hangi disk, hangi serial port). Linux SCSI disk icin: $minor = 16 * drivenum + partition$

- Aygıt dosyaları `mknod` komutuyla oluşturulur.

[S] Sınav

Sınav sorusu tipi: `ls -l` çıktısında `crw-w-- 1 136, 6 /dev/pts/6` görüyorsunuz. Bu satır ne anlama geliyor? – Major=136, Minor=6, character device, sadece sahibi okuyabilir/yazabilir, grup yazabilir.

1.4 Aygıt Özge Semantikler ve `ioctl`

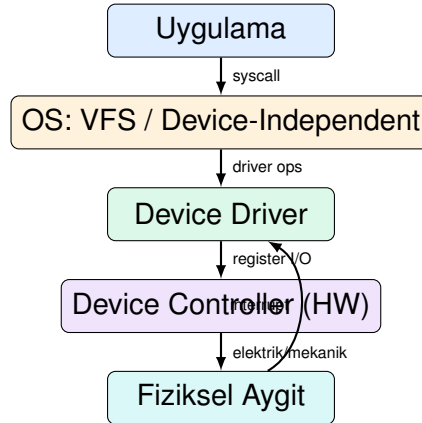
Bazı aygıtlar standart `open/read/write/close` sistem çağrılarına tam olarak uymayan özelliklere sahiptir.

- **`ioctl`**: Aygıt özge kontrol komutu. Teyp sürücüsünde "bir dosyayı atla" gibi komutlar `ioctl` ile gönderilir.
- **Line discipline** (tty): Terminal benzeri aygıtların tamamında devreye giren özel işlem katmanı. Backspace, `^C`, `^Z` gibi özel karakterleri işler; *canonical mode*'da tam satırları tampon eder.

2 Aygıt Sürücüsü Mimarisi

2.1 Sürücünün Standart Giriş Noktaları

Her aygıt sürücüsü belirli standart giriş noktalarını (entry points) implement etmek zorundadır: `open`, `close`, `read`, `write`, `ioctl` ve varsa `mmap`, `poll`, `llseek` gibi ek fonksiyonlar. Bu standart arayüz sayesinde OS, herhangi bir sürücüyü aynı biçimde çağırabilir.



2.2 Block Aygıtlar ve Buffer Cache

Block aygıtların birçok özelliği vardır:

- Birincil işlev: blok okuma/yazma ("*give me N blocks starting at block M*")
- **Buffer cache** (tampon bellek) ve bellek altsistemiyle entegre edilir
- Uygulamalar genellikle dosya sistemi üzerinden erişir; doğrudan erişim nadirdir (bazı yüksek performanslı veritabanları dışında)
- Aygıt özge komutlar hala gereklidir (Linux'ta `hdparm` komutu)

2.3 Uniform Arayuz: Standart Surucusuz vs Standartli

Standart Surucu Arayuzu OL- MADAN	Standart Surucu Arayuzu ILE
Her yeni aygit icin OS cekird- eginin degistirilmesi gerekir. Her surucunun OS ile özel baglantisi vardır. Tasinabilirlik dusuktur.	Yeni aygitlar icin sadece yeni surucunun yuklenmesi yeterlidir. Surucu, belgelenmus standart arayuzu implement eder. Cekirdek degismez.

[*] Onemli

Standart surucu arayuzu (uniform interface) sayesinde OS, hicbir aygiti tanimamasina ragmen o aygitla konusabilir. Bu, plug-and-play'in temel prensibidir.

3 Grafik Aygitlari

3.1 Basit Bitmap Modeli ve Donanim Yardimi

Ekrani piksel dizisinden olusan bir bellek tamponudur; her piksel birkac bit ya da bayta karsilik gelir. Yazilim her pikselin degerini hesaplar ve dizinin adresini ekrana bildirir. Bu yaklasim basit ama pratikte son derece yavas kalir.

[>] Ornek

1920x1080, 60 fps: $1920 \times 1080 \times 60 \approx 124$ milyon piksel/saniye. Her piksel 4 bayt (RGBA) ise ≈ 497 MB/s yalnizca yazma gerekir; 3D golgelem hesaplar bunun uzerine gelir. CPU bununla tek basina basa cikamazdi.

Cozum: **Donanim yardimi (hardware assist)**. Kullanici programlari daha yuksek-seviyeli terimlerle calisir: karakterler, poligonlar, doldurulmus alanlar, imle cc, kayan pencereler. Her grafik kontrolu bu yuksek-seviyeli komutlari islemek uzere programlanir – bazi standardlastirilmis (OpenGL, DirectX), bazi ozgun (proprietary) bir dil kullanir. Grafik islemcileri (GPU) cok uzmanlasmis birimlerdir; siralama ve sifreleme gibi genel amaclislerde de kullanilmistir.

OS, grafik surucusunun aygita yuksek-seviyeli komutlar gondermesine izin ver-melidir.

3.2 Pencereler ve Programlama Arayuzu

[*] Onemli

Ekrani paylasilan bir kaynaktır; OS bu payi yoneticir. Temel soyutlama: **Pencere (Window)**. Pencereler ic ice yuvalanabilir. Bir uygulamanin penceresine yazdigi cizimler pencere sinirinda **kirilir (clipped)** – pencerenin disina cikamaz. OS baslik cubugu, simgeler gibi alanlar icin ekranin bir bolumunu kendisi kullanir. (Bu konu bu ders icin kapsam disindadir; ancak kavramlarin bilinmesi beklenmektedir.)

4 Ag Aygitlari (Network Devices)

4.1 Temel Farklar

Disk suruculere benzer sekilde ag aygitlari cok sayida surecin paylasimindadir; ancak kritik farklari vardır:

Ozellik	Disk	Ag Aygiti
Organizasyon	Dizin, FAT var	Dizin/FAT yok
Erisim modeli	Block / Character	Paket tabanlı
Giris tetigi	Istege gore	Istenmeden (unsolicited)
Hiz kontrolu	Kontrol edilebilir	Kontrolsuz
Kimlik tanımlama	Dosya tanıtıcı	Paket içerisinde işaretleme

4.2 Ag Cikisi

Cikis paketleri gorece kolaydir: Kullanıcının dosya tanıtıcısından elde edilen bilgiye gore paket isaretleme. Suruculer yerel donanima uyum saglamalidir:

- **Ethernet:** 32-bit IP adresini 48-bit Ethernet adresine donustur – ARP (Address Resolution Protocol) kullanilir
- **Dial-up (PPP):** Paket cerceveleme (framing) saglanmalidir; PPP her paketi baslik + sona veri cercevesi ile sarar

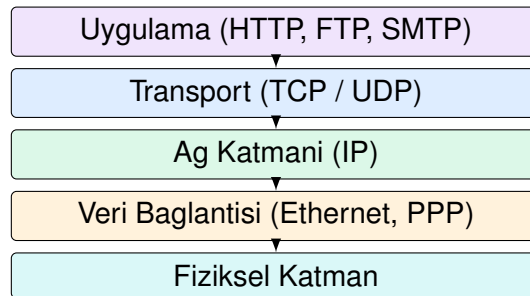
4.3 Ag Giris

Rastgele gelen paketler alinabilir – disk'ten farkli olarak giris *istenmeden* gelir:

- Paketi hangi surecle iliskilendirebilmek icin paket isaretleme kullanilir
- Hiz keyfi yuksek olabilir ve **yavaslatamaz siniz**
- Surucunun paket bufferlarini hizla islemesi gerekir; aksi halde paketler duser

4.4 Ag Yigiti (Network Stack)

Ag isleme cok katmanlidir; OS bu katmanlar arasindaki baglantilari saglar.



Unix System V her ag aygiti ve protokol katmani icin dosya sistemi girislerini kullanirdi. Cogu modern sistem ag aygitlari icin dosya sisteminden bagimsiz bir ad uzayi kullanir.

5 Gercek I/O Sorunlari ve Surucu Ornekleri

5.1 Fiziksel Dunya Tuhafliklari

Aygıt suruculeri fiziksel dunyanin tuhafliklarini tolere etmek zorundadir. Suruculer basit soyut modelden gercek donanim arasindaki mapenin buyuk cogunlugunu gerceklestirirler.

[!] Dikkat

Gerçek surucu yaziminin zorlugu: Donanim hatalari, belirsiz protokoller, yanlış belgeler, gevshek kablolardan kaynakli garip sinyaller ve onlarca farkli donanim revizyonu ile uyumluluk gerektirir.

5.2 Linux Yazici Surucusu Yorumu (drivers/char/lp.c)

Listing 2: Linux lp.c surucusunden gercek yorum

```

1 The so called 'buggy' handshake is really the well documented
2 compatibility mode IEEE1284 handshake. They changed the well
3 known Centronics handshake acking in the middle of busy
4 expecting to not break drivers or legacy application, while
5 they broken linux lp until I fixed it reverse engineering the
6 protocol by hand some month ago...
```

Standart oldugu iddia edilen IEEE 1284 protokolunun bile farkli uretici firmalar tarafından farkli bicimde implement edildiği ve surucu yazarinin protokolu *tersine muhendislikle* anlamak zorunda kaldigi gorulmektedir.

5.3 NetBSD Yazici Surucusu Test Yorumu

Listing 3: NetBSD printer driver test talimatlarindan

```

1 1. Write/read back same value to data port.
2   Do alternating zeros, alternating ones, walking zero,
3   and walking one test to check for stuck bits.
4 2. Write/read back same value to control port lower 5 bits;
5   upper 3 bits are reserved per IBM PC technical reference
6   manuals and different boards do different things with them.
```

5.4 Hata Kurtarma Kodunu Test Etmek (Slayt 67)

[S] Sinav

Hata kurtarma kodu nasıl test edilir? Pratik sorular:

- Tam dogru miktarda “kotu” medyayi nereden bulacaksınız?
- Hata modlari tamamen keyfi (arbitrary) olabilir
- Gevshek kablolar cok tuhaf sinyaller uretebilir
- Hata kurtarma yollari nadiren yurutur – dolayisiyla test edilmesi zordur

Bu durum, ozellikle guvenlik-kritik sistemlerde buyuk endise kaynagidir.

6 I/O Gerceklestirme Yontemleri

CPU'nun I/O transferini nasıl koordine ettiği bakımından uc temel yaklasim vardır.

6.1 Programmed I/O (Dongusel Sorgulama – Polling)

CPU, veriyi bayt bayt aygita kopyalar ve aygit hazır olana kadar dongu icinde bekler.

Listing 4: Yazici icin Programmed I/O ornegi

```

1 copy_from_user(buffer, p, count); /* kullanicidan kernel'a
   kopyala */
2 for (i = 0; i < count; i++) {
3     while (*printer_status_reg != READY); /* bos dongu -- mesgul
       bekleme */
4     *printer_data_reg = p[i];
5 }
6 return_to_user();
```

Dezavantaj: CPU, I/O tamamlanana kadar bos dongu icerisinde bekler (busy waiting). Bu sure boyunca CPU baska is yapamaz – CPU suresi israf edilir.

6.2 Interrupt-Driven I/O (Kesme Tabanlı)

CPU bir karakter gönderir, ardından başka iş yapmak üzere devam eder. I/O tamamlanınca donanım bir kesme (interrupt) üretir; CPU bu kesmeyi işleyerek sonraki adımı atar.

Listing 5: Interrupt-driven yazıcı – sistem çağrı tarafından çalıştırılan kod

```
1 copy_from_user(buffer, p, count);
2 enable_interrupts();
3 while (*printer_status_reg != READY);
4 *printer_data_reg = p[0]; /* ilk karakteri gönder */
5 scheduler();             /* surecy blokla, baskasini calistir */
```

Listing 6: Interrupt service procedure (kesme servis yordamı)

```
1 if (count == 0) {
2     unblock_user(); /* kullaniciyi uyandur */
3 } else {
4     *printer_data_reg = p[i++];
5     count--;
6     acknowledge_interrupt();
7 }
```

Avantaj: CPU boş beklemesiz. **Dezavantaj:** Her karakter için bir kesme oluşur; yüksek hızlı aygıtlarda çok fazla kesme yükü oluşur.

6.3 DMA (Direct Memory Access – Doğrudan Bellek Erisimi)

CPU, aygıtın bir bellek adresi ve bayt sayısı verir; DMA kontrolörü veriyi doğrudan RAM ile aygıt arasında tasir. CPU yalnızca transfer tamamlanınca bir kez kesme alır.

Listing 7: DMA ile yazıcı – sistem çağrı kodu

```
1 copy_from_user(buffer, p, count);
2 set_up_DMA_controller(p, count); /* adres + sayac ayarla */
3 scheduler();                     /* surecy blokla */
4 /* DMA bitince kesme gelir -- interrupt handler kullaniciyi
   uyandırur */
```

Cycle stealing: DMA kontrolörü, veri tasirken bus'ı kullanan CPU'dan bus döngülerini "capar". CPU bunu hisseder; hafif yavaşlar ancak bloklanmaz.

Ozellik	Programmed I/O	Interrupt-Driven	DMA
CPU mesgul bekleme	Evet	Hayir	Hayir
Kesme sayısı	0	Her karakter	Transfer basi
Kullanım alanı	Dusuk hizli	Orta hiz	Yuksek hizli
Donanim gereksinim	Minimal	Interrupt ctrl	DMA chip

[S] Sinav

Klasik soru: DMA'nin "cycle stealing" ifadesi ne demektir? – DMA kontrolörü, veriyi bellek ile aygıt arasında tasirken bus'ı kullanir. Bu esnada CPU da bus'a erismeye calisabilir ancak DMA oncelik ahr; CPU bir veya bir kac clock dongusu beklemek zorunda kalir. Bu "calinan" donguler cycle stealing olarak adlandirilir.

7 Bellek-Eslemeli I/O (Memory-Mapped I/O) ve IOMMU

7.1 MMIO Mimarileri

Aygıt kayıtlarına (register) erişmek için iki temel yaklaşım vardır:

Yaklaşım	Acıklama	Kullanım
Ayrı I/O Uzayı	Özel I/O komutları (IN/OUT). Aygıt kayıtları farklı bir adres uzayında	x86 eski miras
Memory-Mapped I/O	Aygıt kayıtları bellek adres uzayına eslenir. Normal MOV komutuyla erişim	Modern sistemler
Hybrid	Her iki yaklaşım birlikte	Cogu modern PC

Tek bus mimarisi: CPU, bellek ve I/O aygıtları tek bus paylaşır. Basit ama geniş bant darboğazına neden olabilir.

Çift bus mimarisi: Bellek için ayrı, I/O için ayrı yüksek hızlı bus. Daha performanslıdır.

7.2 IOMMU

IOMMU (Input/Output Memory Management Unit), DMA yetenekli I/O aygıtları için sanal adres dönüşümü sağlar. DMA'nın yalnızca izin verilen fiziksel bellek bölgelerine erişebilmesini sağlar.

[=] Formül

IOMMU ne yapar: Aygıt tarafından görünen sanal adresler → (muhtemelen parçalı) fiziksel adresler. Böylece DMA kontrolörü bitişik sanal adreslerle çalışırken asıl fiziksel bellek parçalı olabilir.

Avantajlar:

- Fiziksel olarak bitişik olmayan büyük bellek bölgeleri tahsis edilebilir (scatter-gather listelerine gerek azalır)
- 32-bit aygıtlar PAE/64-bit fiziksel bellek uzayına erişebilir (bounce buffer gerekliliği ortadan kalkar)
- Güvenlik: Aygıt yalnızca kendisine eslenmiş belleğe erişebilir; DMA saldırılarını engeller

Dezavantajlar:

- Donuturma ve sayfa tablosu yurumeleri nedeniyle performans düşebilir
- I/O sayfa tablosu için ek fiziksel bellek gerekir

8 Kesme Yonetimi (Interrupt Management)

8.1 Kesme Nasıl Olur?

Bir I/O işlemi tamamlandığında aygıt kontrolörü, interrupt controller'a bir sinyal gönderir. Interrupt controller, CPU'ya INT sinyali atar. CPU mevcut işlemi bitirip (ya da terk edip) kesme servis yordamını (Interrupt Service Routine – ISR) çağırır.

8.2 Precise vs Imprecise Interrupts

[#] Tanım

Precise Interrupt (Kesin Kesme): Kesmenin nerede oldugunu tam olarak bilinebilir. 4 ozellik saglanmalıdır:

1. PC (Program Counter), bilinen bir yerde kaydedilmiştir
2. PC'nin gosterdigi komuttan önceki **tüm** komutlar tamamiyla yurutulmustur
3. PC'nin gosterdigi komutun **otesindeki** hicbir komut yurutulmemistir
4. PC'nin gosterdigi komutun yurutme durumu bilinmektedir (yarida mi, baslamadi mi?)

Imprecise Interrupt (Kesin Olmayan Kesme): Boru hattı (pipeline) nedeniyle farklı yurutme asamalarında birden fazla komut bulunabilir. Kesme anında hangi komutların bitirildigi, hangilerinin bitmedigini soylemek zordur. Çok sayfalık kayıt kaydetme ve geri yukleme gerektirdigi için kesme islemini yavaslatan karmasik bir durumdur. x86 precise, bazı RISC mimarileri imprecise kullanır.

[S] Sinav

Sinav sorusu: Precise interrupt'in 4 ozelligini siralayiniz ve bunların neden önemli oldugunu aciklayiniz. *Cevap ozeti:* Bu ozellikler, kesme anındaki islemci durumunun tutarli sekilde kaydedilmesini ve geri yuklenmesini saglar. Context switch (baglam anahtarlama) ve page fault iyilestirmesi için kritiktir.

8.3 Kesme Yuku ve Coalescing

Yuksek I/O hizlarında (gigabit ag, yuksek hizli SSD) her I/O tamamlanmasi için bir kesme olusturmak CPU'yu asabilir.

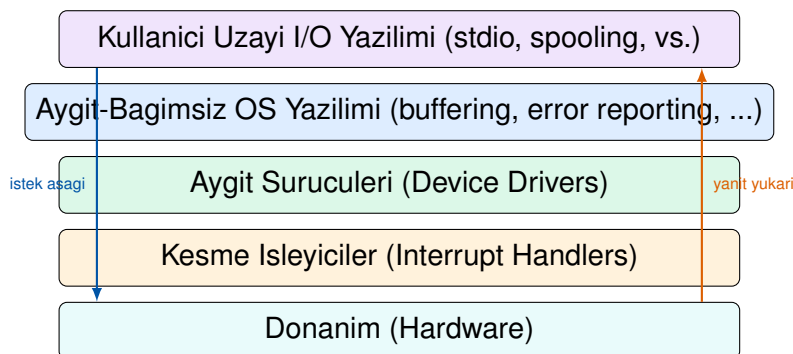
Coalescing (Birlestime): Aygit, her n. I/O tamamlanmasında bir kesme uretir. Bu sayede kesme sayisi n kat azalir; ancak ilk n-1 islem için gecikme artar.

Polling vs Interrupt trade-off:

- Dusuk I/O orani: Interrupt avantajlidir (CPU yalnızca gerektiginde uyandırılır)
- Çok yuksek I/O orani: Polling (spin loop + sleep) daha verimli olabilir cunku kesme yuku asabilir

9 I/O Yazılım Katmanlari

I/O yazilimi dört ana katmanda organize edilir; her katman bir alt katmana hizmet verir.



9.1 Kesme Isleyici – 10 Adim

Bir kesme geldiginde yapilmasi gereken 10 adim:

1. Kesme donanimi tarafından henuz kaydedilmemis kayitcilari kaydet
2. Kesme servis yordami icin bir baglam (context) kur
3. ISR icin bir stack kur
4. Interrupt controller'i onayla (acknowledge); merkezi interrupt controller yoksa kesmeleri yeniden etkinlestir
5. Kayitcilari bulundugu yerden surecin process table girisine kopyala
6. ISR'i calistir
7. Sirayi alacak sureci sec
8. Calisacak surecin MMU baglamini kur
9. Yeni surecin kayitclarini, PSW dahil yukle
10. Yeni sureci calistirmaya basla

[!] Dikkat

Bu 10 adim yalnızca kesme geldiği zaman değil, context switch gerçekleştiği her durumda da dikkate alınmalıdır. Adımlar genellikle dört şeye odaklanır: kayıt alma, ISR çalıştırma, surecim secimi, baglam geri yukleme.

9.2 Aygit-Bagimsiz I/O Yazilimi

Bu katmanın görevi suruculere standart bir arayuz saglamak ve ust katmanlara tekduzre bir hizmet sunmaktır. Tipik islevler:

- Suruculer icin tekduzre arayuz saglamak
- Tamponlama (buffering)
- Hata raporlamasi
- Ozel/surucuye ozel olmayan aygitlari tahsis etmek ve serbest birakmak
- Suruculere bagimsiz bir blok boyutu saglamak

9.3 Arabellekleme (Buffering)

Mod	Nasil Calisir	Nerede Kullanilir
Tampon yok	Veri dogrudan kullanıcı bel- legine	Cogu block I/O
Kullanici uzayi tamponu	Veri kullanıcının tampon- nunda birikir	stdio
Kernel tamponu	Kernel'da birikir, sonra kopyalanir	Ag, disk
Cift tampon (double buffering)	Bir tampon dolarken diger bosaltilir	Yuksek hizli akis

Ag tamponlamasi: Bir ag paketi; ag kart surucusu, ag katmani, transport katmani, uygulama gibi katmanlar boyunca birden fazla kez kopyalanabilir. Bu kopyalar bazen kacinilamazdir.

[S] Sinav

Soru: Double buffering'in amaci nedir? – Bir tampon I/O aygiti tarafından doldurulurken, diger tamponun CPU tarafından islenmesine izin verir. Bu sayede I/O ve hesaplama paralel yurutur; "bos bekleme" suresini azaltir.

10 Bellek Erisim Sorunlari ve DMA

10.1 Sanal Bellek Sorunu

Kernel sanal adreslerle calisir; bunlar fiziksel adreslere esit degildir. Bitisik sanal adresler muhtemelen fiziksel olarak bitisik degildir. Cozum yollari:

- I/O map: Bus yoneticisi sanal bellek haritasini bilir
- Kernel, sanal adresleri kendisi donusturur

10.2 Scatter/Gather I/O

Tek bir paket ya da disk blogu, fiziksel bellekte bitisik olmayan iki veya daha fazla sayfaya okunacaksa, I/O transferinin birden fazla {adres, uzunluk} cifti kullanmasi gerekir. Okumada "scatter" (dagit), yazmada "gather" (topla) islemi yapilir.

10.3 Direct I/O ve Sayfa Kilitleme

Performans icin veriyi kullanıcı uzayina kopyalamaktan kacinilebilir (zero-copy). Dogrudan I/O mumkun olmasi icin:

- Kullanici sanal bellek haritasina bakilmalidir (adres donusumu)
- I/O suresince sayfalar fiziksel bellekte *kilitlenmelidir* (page locking); aksi halde sayfa silinebilir

10.4 Bounce Buffer

Eski I/O aygitlari ve bus'lar, sistemin tum bellegini adreslemes yapamayabilir.

[>] Ornek

Ornek: ISA bus 16-bit adres kaydedicisine sahiptir – yalnızca 64 KB adresleyebilir. Bircok eski PCI aygiti fiziksel adres uzayinin 4 GB'inin otesini adresleyemez. Cozum: Izin verilen dusuk bellege (bounce buffer) I/O yap, ardindan sayfayi kopyala. Genellikle sayfa yeniden eslenemez cunku o kiymetli dusuk bellekalani gereklidir.

10.5 Disk Blok Adres Uzayi – Tarihsel Sinirlar

Sinir	Neden
504 MB	CHS için maksimum deger
1.97 GB	BIOS'ta 12-bit silindir boyutu siniri
2 GB	En büyük FAT16 diski
3.94 GB	16 bas ile en fazla 8K silindir
7.38 GB	8K silindir sinirini asabilmek için en fazla 240 bas
7.88 GB	BIOS CHS adreslemeyle 7.88G'in ötesini destekleyemez; LBA gerekir
128 GB	Disk spesifikasyonu 28-bit sektor numarasi kullanir
2.2 TB	Windows XP siniri

11 Disk Zamanlama Algoritmaları

11.1 Disk Erisim Suresi Bilesenleri

[=] Formül

Disk Erisim Suresi = Seek time + Rotational delay + Transfer time

- **Seek time:** Kolu dogru silindire tastima suresi
- **Rotational delay:** Dogru sektorun basa gelmesini bekleme suresi (ortalama yarım tur)
- **Transfer time:** Verinin diskten aktarılma suresi

Seek time genellikle en dominant bileşendir.

11.2 Catisan Gereksinimler

Disk zamanlama algoritmaları üç amacı dengelemeye çalışır:

- **Adalet (Fairness):** I/O isteklerini geliş sırasına göre işle
- **Verimlilik (Efficiency):** Kol hareketini minimize et
- **Hız (Promptness):** Hiçbir istegi sonsuz bekletme

11.3 FCFS (First Come, First Served)

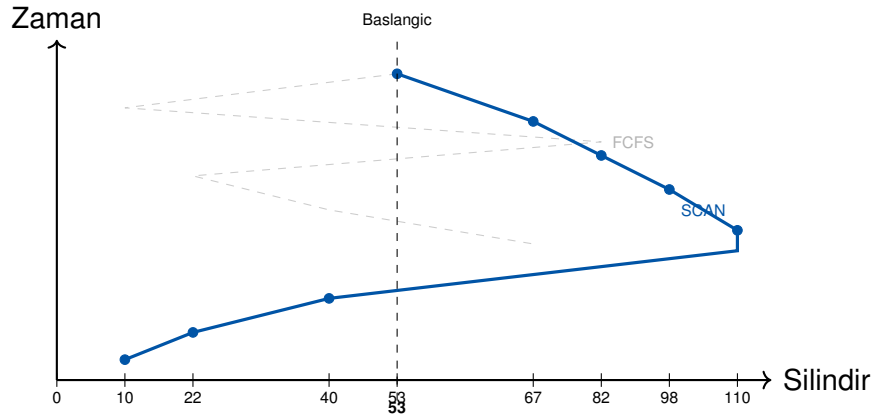
Istekler geliş sırasına göre işlenir. Adildir ancak son derece verimsizdir; kol ileri geri atlayabilir.

11.4 SSF (Shortest Seek First)

Her seferinde mevcut kol pozisyonuna en yakın istegi işle. Verimlidir ancak adil değildir; disk uçlarındaki istekler sonsuz bekleyebilir (starvation).

11.5 Elevator (SCAN) Algoritması

Kol tek yönde hareket eder, o yöndeki tüm istekleri siraya göre karşılar; uç noktaya ulaşınca yönü döndürür.



[!] Dikkat

Elevator algoritmasının hala adalet sorunu vardır: Kol belirli bir noktanın etrafında yoğun istekler alıyorsa (dosya sistemleri blokları yerel tahsis etmeye çalışır), o bölgede gidip gelebilir ve uzak istekler bekleyebilir.

Pratik zorluklar:

- Dosya sistemi güvenlik semantikleri bazı blokların belirli bir siraya yazılmasını gerektirir; bu kuyruğa süreksizlik yaratır
- Gerçek disk geometrisini (silindir, iz, sektör düzenlemesi) bilmek zordur; kontrolör kendi optimizasyonunu yapabilir
- Surucu yeterince eslesli istek göndermeli ki kontrolör de optimizasyon yapabilsin

11.6 Donme Gecikmesi ve Silindir Eskiltmesi (Cylinder Skew)

Art arda seçilen sektörler art arda okunabilmesi için diske etrafında dağıtılabilir. Bu yaklaşım dosya sistemi, surucu veya kontrolör katmanında uygulanabilir.

Cylinder skew: Komsu silindirler arasında geçiş süresi (seek time) dikkate alınarak, komşu silindir aynı numaralı sektörü biraz ilerde başlatır; böylece silindir değişimi tamamlandıktan hemen sonra hedef sektör başa gelir.

12 Manyetik Disk Parametreleri ve Bicimlendirme

12.1 Disk Parametreleri

Aşağıdaki tablo, iki ürün noktasındaki disk parametrelerini göstermektedir (Tanenbaum Tablo 5-3'ten):

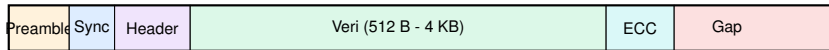
Parametre	IBM PC 360-KB Floppy	WD 18300 Hard Disk
Silindir sayısı	40	10,242
Track / silindir	2	255
Sektör / track	9	63 (ortalama)
Sektör / disk	720	≈ 35.7M
Sektör boyutu	512 B	512 B
Disk kapasitesi	360 KB	≈ 18 GB
Seek time (avg)	77 ms	8.9 ms
Rotasyonel gecikme (avg)	100 ms	5.6 ms (5400 RPM)
Transfer hızı	250 KB/s	≈ 50 MB/s

ZBR (Zone Bit Recording): Fiziksel olarak dis izler (silindirlerin disindeki izler) ic izlerden daha uzundur. ZBR, dis zonlara daha fazla sektor koyarak bu ek kapasiteden yararlanir. Bu nedenle dis zonlarda veri aktarim hizi daha yuksektir.

Sanal Geometri: Gercek (fiziksel) geometri ile BIOS/OS'in gosterdigi geometri farkli olabilir.

- Fiziksel geometri: gercek silindir/kafa/sektor duzeni (ZBR nedeniyle zon basi farkli sektor sayisi)
- Sanal geometri: BIOS'a sunulan tekduzge C/H/S degerleri – sadece adres uzayini kapsar
- Modern diskler **LBA (Logical Block Addressing)** kullanarak bu farklilik soyutlar: her blok 0'dan baslayan sirayla numaralandirilir

12.2 Sektor Yapisi



Interleaving (Sektor Atlama): Okuma/yazma basliginin gerekli islemleri yapabilmesi icin sektor sirasi kasitli olarak atlatli duzenlenir (tek atlama, cift atlama). Modern disklerde artik genellikle gerekli degildir cunku tampon bellek yeterli hiza kavusmustur.

13 Hata Isleme ve Sabit Depolama

13.1 Hata Tipleri

I/O aygitalari ve fiziksel medyalar kusursuzdur – surucular hata kosullariyla basa cikmak zorundadir. Hatalar genellikle ECC (Error Correcting Code) baytlariyla algillanir.

Teyp Hatalari:

1. 10 kez yeniden dene
2. Teypte birden fazla inch geri sar ve “shoe shine” yap: kotu bolgeyi okuma kafasinin kenarina surt
3. Amac: kireci (dirt) temizlemek

Disk I/O Hatalari:

1. Birkac kez yeniden dene
2. Hala basarisizsa bloku "kotu" olarak isaretle (yontem aygita ozgudur)
3. Yedek bir blok tahsis et
4. Yeniden eslesmenin gerceklestirildigi bir sekilde gosterilmeli

13.2 Sabit Depolama (Stable Storage)

Yaz-ve-cokme (write + crash) senaryosunda veri tutarlilikini garantilemek icin her iki disk uzerinde de islem yapilir.

[#] Tanim

Sabit Yazı (Stable Write):

1. diske yaz. Dogrula. Basarili mi? Devam et.
2. 2. diske yaz. Dogrula. Basarili mi? Tamam.

Sabit Okuma: 1. diskten oku; hatali ise 2. diskten oku.

Crash Recovery: Cokme noktasina gore onarim yapilir: ikisi eslesiyorsa tamam; eslemiyorsa 2. diski 1. ile eslestirir ya da tersini yapar.

14 RAID

RAID (Redundant Array of Independent Disks – Bagimssiz Disk Dizisi), hata toleransi ve/veya performans icin birden fazla diski birlikte kullanma yontemidir.

Seviye	Yontem	Okuma	Yazma	Hata Tol.
RAID 0	Striping (serit)	↑↑	↑↑	Yok (daha kotu)
RAID 1	Mirroring (eslemek)	↑	↓	1 disk
RAID 4	Parite surucusu	↑	↓↓	1 disk
RAID 5	Dagitilmis parite	↑↑	↓	1 disk
RAID 6	Cift dagitilmis parite	↑↑	↓↓	2 disk
RAID 10	Stripe + mirror	↑↑	↑	1 disk/grupta

[=] Formül

RAID 4/5 parite: $P = D_1 \oplus D_2 \oplus D_3 \oplus \dots \oplus D_n$ Bir disk cokerse: $D_{lost} = D_1 \oplus D_2 \oplus \dots \oplus P$ (kalan disklerle yeniden olusturulabilir)

RAID 4 sorunu: Tum parite yazmalari tek “parite surucusu”ne yogelir. Bu surucu darbogazdir. RAID 5, parite seritlerini tum suruculere yayarak bu darbogazdani acar.

Implementasyon: RAID kontrolor donaniminda veya OS’te “pseudo-device driver” olarak implement edilebilir.

15 Optik Depolama: CD-ROM, CD-R, DVD

Ozellik	CD-ROM	DVD (tek kat)	DVD (cift kat)
Pit boyutu	0.8 μm	0.4 μm	0.4 μm
Iz araligi	1.6 μm	0.74 μm	0.74 μm
Lazer	0.78 μm (kizilotesi)	0.65 μm (kirmizi)	0.65 μm
Kapasite	650 MB	4.7 GB	8.5 GB

DVD formatlari: (1) Tek yuz/tek kat = 4.7 GB, (2) Tek yuz/cift kat = 8.5 GB, (3) Cift yuz/tek kat = 9.4 GB, (4) Cift yuz/cift kat = 17 GB.

CD-R’de boya tabakasi vardir: laser boya molkulleri yakar, "yanmis" bolge optik olarak CD pit gibi davranir.

16 Saat Donanimi ve Yazilimi

16.1 Saat Donanimini

Programlanabilir saat genellikle bir kristal osilatoru ve programlanabilir bir sayac icerir. Sayac sifira ulasinca interrupt uretir.

16.2 Saat Surucusunun Sorumluluklar

1. **Gunun saatini (time-of-day) tutmak**
2. **Sureclerin izin verilen sureden fazla calismasini onlemek** (zamanlayici tick)
3. **CPU kullanimini hesaba katmak** (user/kernel surelerini ayri say)
4. **Kullanici surecleri tarafından yapilan alarm sistem cagrilarini islemek**
5. **Sistemin kendisi icin watchdog timer saglamak**
6. **Profilleme, izleme, istatistik toplama**

16.3 Gunun Saatini Tutma

Uc yaklasim vardır:

- Her tekte tamsayi artirma (sayac overflow riskli)
- Saniye sayimi ve kesir tutma (iki sayi tutma)
- Baslangic anini kaydet; mevcut zamani hesapla

16.4 Tek Saat ile Birden Fazla Timer

Birden fazla zamanlayici olusturmak icin onceden siralanmis bir liste tutulur. Liste elemanlari *kalan tick sayisi* degil, *bir oncekine gore delta tick* saklar; sifira ulasan elemandan oncekiler cikarilir.

[>] Ornek

Ornek: 10, 25, 35 ve 60 tekte ates edilecek zamanlayicilar icin liste: 10 → 15 → 10 → 25. Her tick'te liste basinin degeri 1 azaltilir; sifira ulasinca o zamanlayici calistirilir ve basta sonraki eleman 1 azaltilmaya devam eder.

16.5 Soft Timer

Her fiziksel kesme ayri bir kesme maliyeti tasir. Soft timer, kesme yerine kernel giris noktalarini (system call, TLB miss, page fault, I/O interrupt, CPU idle) sayar; o noktada zamanlayiciyi kontrol eder. Dusuk ek yuk, makul duyarlilik saglar.

17 Klavye ve Terminal Yazilimi

Terminal yazilimi, klavyeden gelen baytlari islerken “canonical mode” (kanonik mod) gibi özel davranislari destekler.

[*] Onemli

Canonical mode: Tam satirlar tampon edilir ve Enter'a basildikten sonra uygulamaya teslim edilir. Özel karakterler:

- Backspace/DEL: Son karakteri sil
- ^C: SIGINT gonder (surecin olmasina neden ol)
- ^Z: SIGTSTP gonder (sureci dondur)
- ^D: EOF (End of File)
- ^S / ^Q: Akisi durdur / devam ettir

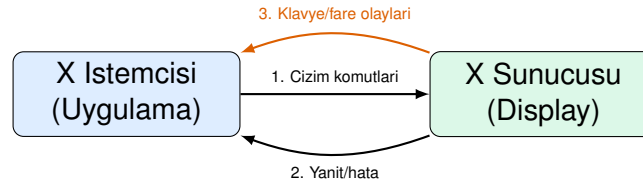
ANSI Kacis Dizileri (Escape Sequences): Terminal ciktisinda imlec hareketi, renk vb.

icin ESC[... biciminde diziler kullanilir. Ornegin ESC[2J ekrani temizler, ESC[A imleci bir yukari tasir.

18 X Window Sistemi ve GUI

18.1 Mimari

X Window, istemci-sunucu (client-server) modelini kullanir; ancak “sunucu” ekranin bulunduгу makinededir, “istemci” ise uygulamanin calistigi makinededir.



4 mesaj turu:

1. Uygulamadan is istasyonuna: Cizim komutlari
2. Is istasyonundan uygulamaya: Sorgu yanitlari
3. Is istasyonundan uygulamaya: Klavye, fare, diger olay duyurulari
4. Hata mesajlari

18.2 GUI: X Window Uygulama Yapisi

Tipik bir X Window uygulama iskeletinde olay dongusu (event loop) vardir:

Listing 8: X Window uygulama iskelet kodundan

```

1 Display *disp = XOpenDisplay(NULL);
2 Window win = XCreateSimpleWindow(disp, ...);
3 XSelectInput(disp, win, ExposureMask | KeyPressMask);
4 XMapWindow(disp, win);
5 while (1) {
6     XNextEvent(disp, &event);          /* bekleme + olay al */
7     switch (event.type) {
8         case Expose:      draw(disp, win); break;
9         case KeyPress:    handle_key(&event); break;
10        case ButtonPress: handle_mouse(&event); break;
11    }
12 }
```

18.3 GUI: Windows Ana Program Yapisi

Windows'ta her pencere bir WndProc (Window Procedure) icerir; sistem her olayda onu cagirir:

Listing 9: Windows WndProc iskelet kodundan

```

1 LRESULT CALLBACK WndProc(HWND hwnd, UINT msg,
2                        WPARAM wParam, LPARAM lParam) {
3     switch (msg) {
4         case WM_PAINT:      do_paint(hwnd); break;
5         case WM_KEYDOWN:    handle_key(wParam); break;
6         case WM_DESTROY:    PostQuitMessage(0); break;
7     }
```

```
7         default:         return DefWindowProc (hwnd, msg, wParam,
8             lParam);
9     }
10    return 0;
}
```

[S] Sinav

X Window vs Windows modeli farki: X Window modeli istemci-sunucu (uygulama ve surucu farkli makinelerde olabilir); Windows modeli yerel mesaj dongusu. Her ikisi de olay tabanlıdır (event-driven): uygulama olaylara tepki verir, kendisi ilkendirme yapmaz.

18.4 Bitmap Islemleri

Rectangle, Line, Ellipse: Temel çizim komutları; GPU ya da grafik surucu tarafından donanim hizlandirilarak çizilir.

BitBlt (Bit Block Transfer): Bir piksel dikdortgenini hizlica baska bir konuma kopyalar.

- Pencereleeri kaydirmada kullanilir (kayan icerik)
- Sprite hareketinde kullanilir (oyun karakterleri)
- Pencere onunde-arkasinda yeniden cizmeye kullanilir

[>] Ornek

BitBlt ornegi: Kullanici bir pencereyi baska bir yere suruklediginde, OS arka plan piksellerini BitBlt ile ondeki konumdan yeni konuma kopyalar; sonra uzerine gelen piselleri cizer. Bu sayede tumunu yeniden cizmek zorunda kalmaz.

18.5 Fontlar

Fontlar karakter anahatlarini (outline) farkli buyukluklerde saklar. Vektoel fontlar (True-Type, OpenType) her boyuta iyi olcekler. Raster (bitmap) fontlar sabit boyuttadir; yakin buyukluklerde piksel bozulmasi gorunur.

18.6 Thin Client (Ince Istemci)

Thin client modeli: Islemci guc sunucu tarafindadir; istemci yalnızca ekranda goruntuleme yapar. THINC protokolu gibi protokoller istemci-sunucu arasinda display komutlarini minimize eder: piksel aktarimi yerine yuksek seviyeli çizim komutları gönderilir (boyama dikdortgeni, metin, imge blogu).

[#] Thin Client

Sunucu tarafında hesaplama yapılır; istemci yalnızca görüntü alır ve klavye/fare girdisini gönderir. X Window bu modeli dogal olarak destekler: X sunucusu ekranın olduğu makinededir, uygulama uzak sunucuda çalışır.

19 Guc Yonetimi (Power Management)

19.1 Donanim Guc Tuketimi

Diz ustü bilgisayarda ana bileşenler ve tipik guc tuketimleri:

Bilesken	Tipik Guc	Not
LCD arka aydinlatma	3–5 W	Zone kapatmayla azaltilabilir
CPU	15–45 W	DVFS ile dinamik olarak dusurulebilir
Bellek (RAM)	2–5 W	Bekleme modunda azaltilabilir
GPU	5–15 W	Mobil GPU'larda daha az
Disk (HDD)	2–5 W	SSD cok daha az tuketir
Ag karti (Wi-Fi)	1–3 W	Bagli degilken tamamen kapatilabilir

19.2 Ekran Bolge (Zone) Yonetimi

Arka aydinlatma tum ekrani bir kerede aydinlatir – buyuk guc tuketimi. Cozum:

- Ekrani *zone*'lara (bolge) bol
- Yalnizca aktif pencerelerin oldugu *zone*'lari aydinlat
- Secili pencere *zone*'a gore tasinirilabilir – pencerenin yeri degismese bile *zone* secimini optimize etmek mumkundur

19.3 CPU Guc Olcekleme – DVFS

[=] Formül

DVFS (Dynamic Voltage and Frequency Scaling):

$$P \propto V^2 \cdot f$$

Voltaji 2'ye bolmek (ki bu cekirdek saatini de 2'ye boler) guc tuketimini **4 kat** azaltir. Bu nedenle modern islemciler voltaj ve frekansı dinamik olarak dusurebilir.

ACPI Durumu	Ad	Guc/Uyandırma
C0	Calisma	Tam guc
C1	Halt	Dusuk guc, aninda uyaniyor
C2	Stop-Clock	Daha dusuk, 2us
C3	Sleep	Minimal, ms-mertebe
S3	Suspend to RAM	Yalnizca RAM acikte
S4	Hibernate	Disk'e kaydet, tamamen kapa

20 Kernel vs. Kullanici Uzayi Karari

20.1 Kernel'e Koyma Kriterleri

[*] Onemli

Aasagidaki durumlardan biri gecerliyse kernele koy:

- Yapilacak tek bir yol olmasi gerekiyorsa (sanal bellek kayitcilarini yuklemek gibi)
- Sistem gercekten calisirken once hazir olmasi gerekiyorsa (disk I/O surucusu)
- Kernel-ozel kaynaklara sik erisim gerekiyorsa (sayfa cercevesi geri kazanim algoritmasi)
- Baska secenek yoksa

20.2 Kullanici Uzayina Koyma Kriterleri

- Onemli performans sorunu yoksa

- Performans isabeti var ama nadiren yapiliyorsa
- Davranisini degistirmek isteyebilirsiniz
- Durdurup yeniden baslatabilirseniz
- Yapilandirma icin dosya kullaniyorsa

Linux'ta pseudo-dosya sistemleri (`/proc`, `/sys`) kernel arayzlerini dosya izin sistemiyle birlestirir: her arabirim dosya gibi gorunur, Unix izin sistemi erisimi kisitilar.

21 Daemon'lar ve Guvenlik

21.1 Daemon Nedir?

[#] Tanim

Daemon: Surekli veya periyodik sistem-geneli islevleri gerceklestirmek icin gozetimsiz calistirilan program. Unix'in gecmisi Multics'e ve Yunan mitolojisindeki *daimon* kavramina dayanir: tanrilar ile insanlar arasinda dogasi olan dogu ustü varlik. Daemon'lar sistem islevlerini yerine getirir ancak isletim sisteminin parçasi degildir.

21.2 Yazici Daemon Ornegi

Kullanici programlarinin dogrudan yaziciya erismesine izin verilmemelidir. Yerine bir yazici daemon'i kullanilir. Neden?

- **Hiz:** Yazici programdan daha yavas
- **Catisma:** Birden fazla program es zamanli yazabilmek isteyebilir
- **Hesap tutma:** Bircok kurumda sayfa basina ucret alinir
- **Guvenlik:** Baska is ile girisimin onlenmesi
- **Etiketleme:** Ciktinin dogru bicimde etiketlenmesi
- **Yonlendirme:** Birden fazla yazici, farkli cikti bicimler

Yazici daemon bileşenleri:

- **Kullanici tarafi (user-facing):** Kullanici tarafından cagrilir; basilacak dosyalar veya ciktii alır
- **Yazici tarafi (printer-facing):** Kullanicilarin gonderdigi istekleri alır; yaziciya kopyalar
- **Yonetim tarafi:** Yazicilari gruplamak, izin vermek, acik/kapali yapmak

21.3 Yazici Daemon'da Guvenlik

- Kullanicilarin daemon dosyalarinin uzerine yazamayacagi garantilenmeli: Normal Unix dosya izinleriyle saglanir
- Kullanici tarafi program *setuid* yapilir
- Basilacak dosyalar: kullanici tarafından okunabilir olmalı (guvenlik), daemon tarafindan da okunabilir olmalı (verimlilik)
- Daemon okuyamazsa dosyayi kopyala; kullanici okuyamazsa hata ver

Baglamak (Link) vs Kopyalamak: Kullanıcının basmak istediği dosyaya link oluşturmak caziptir ama çok zordur:

- Kullanıcının o dosyada okuma izni var mı?
- Yarisma koşuluna (race condition) dikkat et

Dogru link kurma adimlar:

1. Kullanici izniyle mevcut dizini belirle; goreli yollari mutlak yollara cevir
2. Daemon izniyle, korunan bir dizinin altindaki tum kullanicilara yazilabilir bir dizine gec
3. Kullanici izniyle o dizinde dosyaya hard link olustur
4. Kullanici izniyle acilabildigi dogrula
5. Boyut/bicimlendirme kararlarini bu asamada saklama (kullanıcının hala linkle sahibi var)

21.4 Daemon Izinleri: En Az Ayricalik Prensibi

Daemon'ların çalıştığı izin düzeyi kritiktir:

- Bazi daemon'lar login'e izin vermek icin root olarak calisir (ornegin bazi ag dae-mon'lari)
- Daha iyi yaklasim: ozel kullanicilar kullan: lpd, games, named, vb.
- Multics'te cogu daemon sysdaemon veya netdaemon grubunda calisirdi

[*] Onemli

Principle of Least Privilege (En Az Ayricalik Prensibi): Her surec, isini yapabilmek icin gereken minimum izinlerle calistirilmalidir. Bir surecin guvenligi ihlal edildiginde zararının minimumda tutulmasını saglar.

22 NFS ve Dosya Kilitleme

NFS (Network File System) hem kullanıcı-modu hem de kernel-modu birimler kullanır:

- Asil disk I/O: performans nedeniyle kernel icindedir
- **Dosya kilitleme (file locking) kullanıcı-duzeyi daemon'ında yapılır**

Neden lock daemon kullanıcı uzayındadır?

- NFS tasarımı gereği *durumsuzdur (stateless)*: istemci ve sunucu serbetce yeniden baslatılabilir; ag kesintilerine karsi da koruma saglar
- Ancak kilitleme dogasi gereği *durumsaldur (stateful)*
- Lock daemon dosya kilitlerini takip eder; kilitlar kilitlenmis bir diske kaydeder (cokme kurtarimi icin)

23 Sistem Baslatma: init, Calistirma Duzeyleri

23.1 /sbin/init – PID 1

[#] Tanim

/sbin/init: Cekirdek tarafından acilis zamaninda elle olusturulan surec tablosu girdisiyle baslanan **surecin sureci (process 1)**. Tumt sureclerin baslangicindir; sistem kapanirken de tumt surecleri sonlandiran son surectir.

23.2 Sistem Ayarlama (sysinit)

init, calistirma duzeylerini ve girerken yapilacaklari tanimlayan bir yapilandirma dosyasini okur. **Sysinit** soz-yurutme duzeyi: sistem ilk baslatmada yapilacaklar.

- Makine adini ayarla
- /proc ve /sys'i bag (mount)
- Belirli donanimini baslatma (donanim baglama saatini ayarla)
- fsck calistir (dosya sisteminin tutarlilikini kontrol et)

23.3 Calistirma Duzeyleri (Run Levels)

Duzey	Ad	Durum
0	Halt	Sistem kapat
1	Single user	Bakim modu
2	Multi-user	NFS olmadan
3	Full multi-user	Her sey aktif
5	X11	Grafik giris
6	Reboot	Yeniden baslatma

Duzey degistirmek: init, /etc/rc cagirarak duzey degistirirken N numarali duzey icin /etc/rcN.d icindeki tumt dosyalar calistirir.

23.4 /etc/rcN.d Betikleri

- K betikleri: Bir servisi **kapat** (kill). Ornek: K50nfs
- S betikleri: Bir servisi **baslatir** (start). Ornek: S80sendmail
- Sayisal sirayla calistirilir (kucukten buyuge)

[>] Ornek

K50nfs ve S80sendmail icin: Oncelikle 50 oncelikli K betigi (NFS'yi sonlandir) calistirilir, ardindan 80 oncelikli S betigi (sendmail'i baslatma) calistirilir. Boylece bagimliliklari dogru sirada yonetilir.

23.5 Diger Hayati Komutlar

- **login:** Root olarak calisir, setuid degildir. Kullanici adi ve sifre alır; kullanıcı ortamini kurar.
- **X Window:** Unix'te cekirdek parçasi degildir; grafik kartini acma iznine sahip olması nedeniyle genellikle root izni gerektirir (xdm, gdm gibi giris bileşenleri vardır).

- **Shell:** Gerekli kullanıcı kabukları yoktur; birden çok kabuk olabilir (/bin/sh yönetim için gereklidir). Bazen “resmi” kabuklar listesi güvenlik nedeniyle gereklidir.
- **mount:** Dosya sistemlerini kullanılabilir hale getirir
- **fsck:** Cokme sonrası dosya sistemlerini onarır

24 Uygulama Problemleri – Sınav Hazırlık

24.1 Problem 1: DMA Transfer Suresi Hesabi

[>] Örnek

Soru: 8 MHz CPU, 32-bit bus, DMA, 32-bit transferler. DMA 40 ns’de çevrim çalışabilir. 512 KB okunuyor. (a) Transfer kaç çevrim sürar? (b) Transfere kaç ns harcanıyor? (c) CPU bu süre boyunca kaç çevrimi kaybeder?

Cozum:

- 512 KB = 524288 B, 32-bit (4 bayt) transferlerle: $524288/4 = 131072$ çevrim
- Toplam süre: $131072 \times 40 \text{ ns} = 5.24 \text{ ms}$
- 8 MHz CPU çevrim süresi = 125 ns. Kaybedilen çevrim: $5,242,880 \text{ ns} / 125 \text{ ns} = 41943$ çevrim

24.2 Problem 2: Disk Erisim Suresi

[>] Örnek

Soru: Seek time ortalama 8 ms, disk 7200 RPM, 512 bayt/sektor, 300 sektor/iz. Bir sektoru okumak kaç ms alır?

Cozum:

- Seek time: 8 ms
- Rotational delay (ortalama): $(60/7200)/2 = 4.17 \text{ ms}$
- Transfer time: Bir tam tur = $60/7200 = 8.33 \text{ ms}$; bir sektör = $8.33 / 300 = 0.028 \text{ ms}$
- Toplam: $8 + 4.17 + 0.028 \approx 12.2 \text{ ms}$

24.3 Problem 3: Disk Zamanlama Karsistirma

[>] Örnek

Soru: Disk kolu 53. silindirde. İstek sirasi: 98, 183, 37, 122, 14, 124, 65, 67. FCFS, SSF ve Elevator (SCAN) için toplam kol hareketini hesaplayınız.

FCFS: 53→98→183→37→122→14→124→65→67 Hareket: 45+85+146+85+108+110+59+2 = **640 silindir**

SSF (en yakini once, 53’ten basl): 53→65→67→37→14→98→122→124→183 Hareket: 12+2+30+23+84+24+2+59 = **236 silindir**

SCAN (yukari giderken): 53→65→67→98→122→124→183→37→14 Hareket: 12+2+31+24+2+59+146+23 = **299 silindir**

24.4 Problem 4: Precise Interrupt Analizi

[>] Ornek

Soru: Bir surucu pipelineinde A ve B komutlari es zamanli yurutulurken kesme geliyor. A tamamlandi, B bitmedi. Bu precise mi imprecise mi? Sorunlar nelerdir?

Cevap: Imprecise. B komutu ne tamamlandi ne de tam olarak baslamadi – belirsiz bir durumda. Kurtarma (recovery) icin B'nin geri alinmasi ya da tamamlanmasi gerekir. Bu, pipeline'in cikti registrlarini kaydetmeyi zorlastirir ve context save/restore kodunu karmasiklestirir.

24.5 Problem 5: IOMMU vs Bounce Buffer

[>] Ornek

Soru: 32-bit PCI kartli sistemde 6 GB RAM var. IOMMU olmadan DMA nasil calisir? IOMMU ne saglar?

Cevap: 32-bit PCI kart yalnızca 4 GB'a kadar adresleyebilir; 4-6 GB arasi bellege dogrudan DMA yapamaz. Cozum: Bounce buffer – Kernel, 4 GB altinda dusuk bellege DMA yapar, ardindan yuksek belleye kopyalar; bu ek kopya performansi dusurur. IOMMU ile: Kart sanal adres kullanir; IOMMU 4 GB ustundeki fiziksel adreslere donusturur. Bounce buffer gerekliligi ortadan kalkar.

24.6 Problem 6: Yazici Daemon Guvenlik Analizi

[>] Ornek

Soru: Bir kullanıcı kendi dosyasini basmak istiyor. Daemon o dosyaya neden link degil, kopyasini aliyor?

Cevap: Link ile sorunlar: (1) Kullanici dosyayi silebilir veya degistirebilir – race condition. (2) Kullanicinin o dosyada okuma izni olup olmadigini dogrulama karisiktir. (3) Hardlink olusturma sirasinda kullanıcı izni, daemon izni gereksinim sirasi karmasiktir. Kopyalama bu sorunlari cok daha sade bicimde cozer; guvenlik garantisi saglar.

25 Karsilastirma Tabloları

25.1 I/O Yazilim Katmanlari Ozeti

Katman	Ana Islevler	Ornekler
Kullanici uzayi	Spooling, format donusumu	Printf, write sistem ca-grisi
Aygıt-bagimsiz	Tamponlama, hata raporlama, birlesik arayuz	Buffer cache, /dev dosyalari
Aygıt suruculeri	Donanima ozgu kontrol	disk_driver, eth_driver
Kesme isleyiciler	ISR, baglam kaydet/geri yukle	IRQ8 = RTC, IRQ14 = IDE
Donanim	Fiziksel islemler	Disk kolunu hareket ettir

25.2 RAID Secimleri

Senaryo	Onerilir RAID	Neden
Sadece performans, guvenilirlik onemli degil	RAID 0	Maksimum serit genisligi
Yuksek guvenilirlik, okuma yogun	RAID 1	Basit, hizli okuma
Ekonomik, yazma orta	RAID 5	Parite dagitilmis, makul maliyet
Maksimum guvenilirlik + hiz	RAID 10	Stripe + mirror

25.3 I/O Yontemleri Ozeti

	Programmed I/O	Interrupt-Driven	DMA
CPU kullanimi	Tam (bos dongu)	Kesme bos	Minimal (cevrilm calma)
Gecikme	Dusuk (polling)	Orta	Dusuk (sonunda kesme)
Donanim Uygunluk	Yok	Interrupt ctrl	DMA chip
	Dusuk hizli	Orta hiz	Yuksek hizli bulk

26 Kavram Sozlugu

Terim	Aciklama
Block device	Sabit boyutlu bloklara gore erisilen aygit (disk, SSD)
Bounce buffer	Sinirli adres uzayina sahip aygitlar icin dusuk bellege yapilan gecici kopya
Buffering	Veriyi gecici tampon bellekte tutma; hiz farklarini dengelemek icin
Canonical mode	Terminal girisi icin tam satir modlu tamponlama
Character device	Yapisiz bayt akisi sunan aygit (terminal, fare)
Coalescing	N. I/O tamamlanmasi basina bir kesme uretme; kesme yukunu azaltir
Cycle stealing	DMA'nin bus cevrimi kaparken CPU'yu kisa sureyle geciktirmesi
Cylinder skew	Komsu silindirlerde ayni numarali sektorun ofset baslatilmasi
Daemon	Gozetimsiz calistirilan sistem-geneli arka plan sureci
Device driver	Belirli bir donanim turu icin OS ile donanim arasina aracisi yazilim
DMA	Direct Memory Access; CPU katilimi olmadan bellek-aygit veri aktarimi
Double buffering	I/O ve CPU islemini catismadan paralel yurutmek icin iki tampon kullanimi
ECC	Error Correcting Code; hata algilama ve duzeltme icin fazlali bit
Elevator (SCAN)	Disk kolunu tek yone hareket ettirip ucta yonunu ceviren algoritma
Entry point	Surucunun sundugu standart islevler (open, close, read, write, ioctl)

Terim	Aciklama
FCFS	First Come First Served; gelis sirasina gore I/O isleme
Imprecise interrupt	Pipeline nedeniyle kesme anindaki yurutme durumunun belirsiz oldugu kesme
Interrupt coalescing	Bkz. Coalescing
IOMMU	I/O aygitalari icin sanal-fiziksel adres donusumu; DMA guvenligi
ISR	Interrupt Service Routine; kesme geldiginde calistirilan kod
ioctl	Aygita ozgu kontrol sistemi cagirlari
Line discipline	tty aygitlarinda ozel karakter islemesi yapan katman
Major number	Hangi surucunun kullanilacagini belirten aygit numarası
Minor number	Surucu icerisinde hangi birimi belirten aygit numarası
mknod	Unix aygit dosyasi olusturma komutu
MMIO	Memory-Mapped I/O; aygit kayitcilari bellek adres uzayina eslenmesi
Precise interrupt	4 ozellikle tanimlanan, yurutme durumu kesin bilinen kesme
Programmed I/O	CPU'nun bos dongu ile aygiti sorgulayarak veri transfer etmesi
RAID	Redundant Array of Independent Disks; cok diskli depolama
Run level	Unix'te sistemin hangi servislerin aktif oldugunu belirleyen calisma modu
Scatter/Gather	Fiziksel olarak dagit sayfalara I/O yapilmasi
SSF	Shortest Seek First; en yakin silindiri once isleme
Soft timer	Fiziksel kesme yerine kernel girislerinde kontrol edilen timer
Stable storage	Yaz-cokme karsisnda verilerin buyuk olasilikla yazdigi garantilenmiş depolama
Sysinit	Init'in en basinda yaptigi tek seferlik sistem baslangic islemleri
Uniform interface	Tum suruculer icin standart OS arayuzu; yeni aygit = yeni surucu (cekirdek degismez)
ZBR	Zone Bit Recording; dis izlerde daha fazla sektor bulunmasi

27 Sinav Calistirma – Son Revizyon Listesi

1. I/O yaziliminin **4 katmanini** siralayabilmeli ve her birinin gorevini aciklayabilmelisiniz.
2. **Precise interrupt'in 4 ozelligini** ezberleyiniz; imprecise ile farkini aciklayabilmelisiniz.
3. **DMA cycle stealing** nedir, CPU performansina etkisi nedir?

4. Disk erisimini olusturan 3 sure bileseni (seek, rotational delay, transfer) icin **sayisal hesap** yapabilmelisiniz.
5. **FCFS, SSF, Elevator** icin toplam kol hareketi hesaplayip karsilastirabileceginizi dogrulayin.
6. **Unix major/minor device number** mekanizmasini aciklayabilmelisiniz.
7. **Bounce buffer** neden gereklidir? IOMMU ne saglar?
8. **Double buffering** ile ne kazanilir?
9. **RAID 0/1/4/5** arasindaki farklar; XOR parite formulu.
10. **Interrupt handlers 10 adimi** ozetleyebilmelisiniz.
11. **DVFS formulu:** $P \propto V^2 \cdot f$; voltaji 2'ye bolmek guc tuketimi uzerinde ne etkisi var?
12. **Daemon** nedir; yazici daemon'un neden gerekli oldugunu aciklayabilmelisiniz.
13. **Principle of Least Privilege** nedir; daemon izinlerine nasil uygulanir?
14. **NFS lock daemon** neden kullanci uzayindadir? (Stateless vs stateful)
15. **/sbin/init, run levels, /etc/rcN.d K/S betikleri** arasindaki iliskiyi aciklayabilmelisiniz.
16. **Stable storage:** yazi, okuma ve crash recovery islemlerini tanimlayabilmelisiniz.
17. **Canonical mode:** hangi karakterler ozel isle gorur?
18. **X Window 4 mesaj turu** ve client-server mimarisini gosterebilmelisiniz.
19. **Cylinder skew ve interleaving** neden yapilir?
20. Kernele neyin konulmasi gerektigine iliskin **kriteri** aciklayabilmelisiniz.

CSE 312 Isletim Sistemleri – I/O Yonetimi Ders Notu

Prof. Dr. Yakup Genc · GTU Bahar 2026 · Tanenbaum & Bos MOS 5th ed. Bolum 5